A Study of LSTM Optimisation for Forecasting Volatile Time Series Data

Deddy Gunawan Taslim, Hendra Tjahyadi*, I Made Murwantara

Graduate Informatics Dept., Faculty of Computer Science, Universitas Pelita Harapan, Jakarta, Indonesia
Email: deddy.taslim@outlook.com, hendra.tjahyadi@uph.edu, made.murwantara@uph.edu
*correspondence author

2024 4th International Conference on Computer Communication and Artificial Intelligence (CCAI) | 979-8-3503-6276-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/CCAI61966.2024.10603183

***Abstract*—Forecasting time series data is an important subject in economics and business where the Autoregressive Integrated Moving Average (ARIMA) has been extensively used despite its weaknesses, from requiring a minimum number of data points to the assumption of linearity of data. With recent advancement, the Long Short-Term Memory (LSTM) shows potential to address such weaknesses. This research is aimed to identify the effect of hyperparameters such as the number of epochs, units, and optimiser on accuracy of prediction on volatile time series data. Performance metric used is the model accuracy measured with RMSE. This research concluded that the accuracy of LSTM can be improved by increasing the number of epochs, where the increase of the number of epochs from 25 to 125 can increase the prediction accuracy by 64%, and 77% when increased from 25 to 300 epochs. SGDM as the optimiser or solver was able to produce the most accurate result (RMSE at 0.51) compared to Adam and RMSProp (RMSE at 0.89 and 0.78, respectively). Meanwhile the number of units showed inconclusive impact to the prediction accuracy.**

***Index Terms*—ARIMA, LSTM, time series, optimisation methods, hyperparameter**

I. Introduction

Forecasting the time series data - a series of observations listed in the order of time - is an important subject in the field of economics, business, and finance. Many people have been looking into forecasting the stock market, supply and demand, even macroeconomic indicators from historical patterns or experiences [1], [2].

Traditionally, statistical models such as the Autoregressive Integrated Moving Average (ARIMA) have been frequently used to forecast time series data for its advantage of easier to understand and to compute mathematically. Yet ARIMA is also known to be suffered from certain conditions such as the need of certain number of data points to obtain accurate predictions, which are not always feasible in business data, and to the assumption of linearity and stationarity of the data [3], and therefore being unable to predict well for highly volatile data.

With the recent development and advancement in computational powers, and improvements in advanced machine learning algorithms, Deep Learning method [4] like the Long Short-Term Memory (LSTM) shows a lot of promises and potential to address the time series forecasting problems aforementioned [5].

There have been several ARIMA and LSTM forecasting applications and comparison studies [6]–[9]. Further on LSTM hyperparameters optimisation, Siami-Namini et al. experimented with training times or epochs in LSTM [10], and Chakraborty studied the effect of the increased number of units to the forecasting performance [11]. In term of optimisation algorithm, there has been multiple research resulting in algorithms such as Stochastic Gradient Descent with Momentum (SGDM) [12], Root Mean Square Propagation (RMSProp) [13], and Adaptive Moment Estimation (Adam) [14]. Nevertheless, the author found no study that compared the effect of such hyperparameters optimisation or possible combined effect of hyperparameters optimisation to the improvement in prediction accuracy of LSTM.

Motivated by the researcher's interest in a more advanced Deep Learning method such as LSTM in forecasting time series data, this research is aimed to experiment and explore how hyperparameters of LSTM can be optimised to produce a more accurate result. The rest of the paper is organised as follows. Section II provides the literature study and review of the background knowledge on the topics of interest, section III outlines the research method in modelling and experiment, section IV discusses the result and analysis, and lastly section V concludes the research and provides possible future work.

II. Literature Study

This section reviews the background knowledge of the ARIMA and LSTM techniques used and studied in this research, method comparison and related studies.

A. Auto-Regressive Integrated Moving Average (ARIMA)

ARIMA, initially proposed by Box and Jenkins in 1970 [15], is a generalised model combining the Autoregressive (AR) process and Moving Average (MA) processes to model the time series [10]. The Box-Jenkins approach is very famous for its useful three-stages iterative cycle used for identification, estimation, and verification of time series data. With the advancement of the computer, ARIMA is even further popularised and worked into variations e.g., through univariate ARIMA models, transfer function (dynamic regression) models, and multivariate (vector) ARIMA [16]. As acronym indicates, ARIMA captures the key elements of the model:

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on November 14,2024 at 15:15:09 UTC from IEEE Xplore. Restrictions apply.

- AR: Autoregression. A regression model that measures a certain dependency of an observation to a number of lagged observations (p).
- I: Integrated. A mechanism to improve the stationarity of the time series data by measuring the differences of observations at different times (d).
- MA: Moving Average. A method to evaluate impact to the residual error when a simple moving average is modelled to the lagged observations (q).

The ARIMA model that computes time series has the equation [17]:

$$\begin{aligned} y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} \\ + \epsilon_t - \theta_1 \epsilon_{t-1} - \theta_2 \epsilon_{t-2} - \cdots - \theta_q \epsilon_{t-q} \end{aligned} \tag{1}$$

where y_t and ϵ_t are the current value and random error at time stamp t, respectively; ϕ_i (i = 0, 1, 2, ..., p) and θ_j (j = 0, 1, 2, ..., q) are the model parameters and c as constant or intercept. p and q are orders of the model and are both integers. If q = 0, then Eq. (1) transforms into an AR model of order p. When p = 0, the model becomes MA model of order q. I model of degree d represents the number of times the data have been replaced with the difference between the current and previous values.

B. Long Short-Term Memory (LSTM)

LSTM is a variation of recurrent neural network (RNN) development, first introduced in 1997 [18], that tries to mimic the brain's learning process by remembering what it has seen and differentiate important problems from the unnecessary. LSTM has been used for dynamic system modelling in diverse application areas such as image processing, speech recognition, time series prediction, manufacturing systems, autonomous systems, and measuring energy consumption [19]. A simple LSTM network illustrated in Figure 1 consists of three different gates, i.e., input gate, forget gate, and output gate:

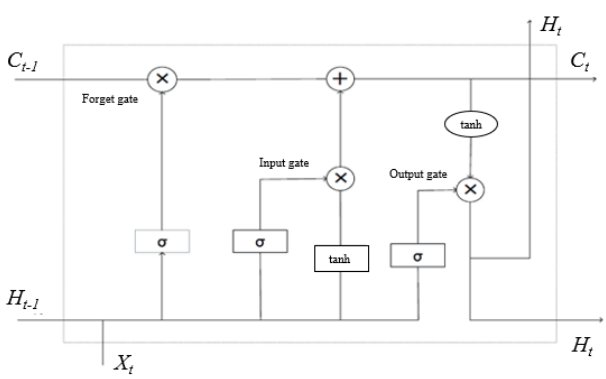

Fig. 1. LSTM Cell Structure [9]

where,

- C_{t-1} = The memory of the previous LSTM cell (at timestamp t – 1)
- C_t = The memory at the current timestamp
- H_{t-1} = The output of the previous LSTM cell (at timestamp t – 1)
- H_t = The output at the current timestamp
- X = Multiplication function
- $+$ = Addition function
- σ = Sigmoid function
- $\tanh$ = Hyperbolic tangent function
- X_t = The input at the current timestamp

There are multiple hyperparameters, explicitly specified parameters that control the training process, that can be used to finetune LSTM as a Deep Learning network. In this paper, the researcher focused on the three often used i.e., epoch, unit, and optimisation algorithm.

Epoch is the total count of the number of times that the dataset is used for training purposes [10]. Therefore, one epoch specifies that the train dataset is submitted to the model once where the data is passed forward and backward while adjusting weight of its network parameter. As Deep Learning algorithms typically use gradient descent to optimise their model, it is logically beneficial to pass the dataset through the Deep Learning network several times to obtain a more accurate model.

An unit is the smallest structure of the LSTM network that contain the LSTM cell itself, input gate, forget gate, and output gate. Chakraborty studied that the increased number of units may improve the prediction result [11].

Neural network is often viewed as an optimisation problem where it seeks to find the global optimum through certain training trajectory and convergence using gradient descent algorithms while also avoiding local minima trap, vanishing gradient, and other challenges [13].

One of the basic approach is the stochastic gradient descent with momentum (SGDM). SGDM incorporates past gradients in each dimension and maintains a momentum in each dimension as a function of previous momentum and current gradient in that dimension. SGDM addresses the weakness of SGD by developing high "velocity" in dimension that has a consistent gradient [12].

Another widely used gradient descent algorithm is the root mean squared propagation (RMSProp), which is an improvement to address the diminishing learning rate of AdaGrad by accumulating the gradient as an exponentially decaying average of squared gradients [13].

A more recent and popular gradient descent variant is the adaptive moment estimation (Adam) [14]. Adam computes adaptive learning rates for each parameter by utilising first moment, that it accumulates the exponentially decaying average of past gradients or similar to SDGM, and second moment where it accumulates the exponentially decaying average of the square of past gradients or similar to AdaDelta and RMSProp.

C. Method Comparison and Related Studies

ARIMA and LSTM models are commonly used for time series forecasting, but the two are intrinsically different. The ARIMA model is simple, easy to interpret, and easy to use, but it does not work well on non-linear time series; meanwhile

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on November 14,2024 at 15:15:09 UTC from IEEE Xplore. Restrictions apply.

the LSTM model is more complex and difficult to interpret, but it is good in processing non-linear time series data [7].

There have been several ARIMA and LSTM forecasting applications and comparison studies [6]–[9]. Joosery and Deepa experimented with lengths of data and noted that the results of LSTM prediction are more satisfactory when a lesser time span of three months of the past data is used as training data. The result worsens when a larger time span of data is used [20]. Siami-Namini et al. compared ARIMA and LSTM based on rolling forecasting origin, with financial time series data, and reported no improvement when the number of training times or epochs was changed [10], and Chakraborty studied that the increased number of units may improve the prediction result [11]. In term of optimisation algorithm, there has been multiple research resulting in algorithms such as Stochastic Gradient Descent with Momentum (SGDM) [12], Root Mean Square Propagation (RMSProp) [13], and Adaptive Moment Estimation (Adam) [14] with each to have shown enhanced effect over the other preceding algorithm.

III. Modelling and Experiment

With the goal of comparing the performance of ARIMA and LSTM, specifically whether fine tuning the deep learning network can improve the overall prediction, the author conducted a series of experiments on the selected financial time series data.

A. Dataset

The research started with the base dataset of Indonesia's monthly consumer price index (CPI) from January 2003 to December 2021 that consist of 228 data points, sourced from Bank of Indonesia (https://www.bi.go.id/id/statistik/indikator/data-inflasi.aspx).

The dataset was cut to appropriate length to simulate the short dataset labelled cpi_36 (36 data points, CPI from January 2005 to December 2007) and long dataset labelled cpi_228 (228 data points, CPI from January 2003 to December 2021).

The base dataset is also intrinsically volatile as can be visually observed from sudden and spike of significant value changes as depicted in Figure 2. Further measure of the volatility in the dataset could also be observed from its coefficient of variation. Coefficient of variation is the ratio of the standard deviation to the mean, which indicates the distribution of data points around the mean which may also indicate volatility in data [21]. Coefficient of variation for both dataset was measured at 45% and 59% for cpi_36 and cpi_228, respectively.

All datasets were also split into train (in-sample) and test (out-of-sample) dataset on a 70:30 ratio.

B. Performance Metric

Performance metrics measured in this research is the prediction accuracy measured in root mean square error (RMSE). RMSE is used for the evaluation of the proposed models. Mean absolute percentage error (MAPE) is not preferred as it is sensitive and potentially biased when the actual data contain very small or close to zero value.

Fig. 2. Indonesia's Consumer Price Index

RMSE computes the root mean of the squared difference of the actual and predicted value. The RMSE metric is very much related to the sum of squared errors and the mean square of errors. For the purpose of model accuracy measurement, these metrics are very useful.

RMSE is defined as follows:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - x_i)^2} \tag{2}$$

Where n is the total count of observations, y_i the actual value; and x_i is the predicted value. RMSE calculates the global error between the actual and predicted value and it is also a quadratic scoring that measures the magnitude of error very well as it punishes large errors [22].

C. Experiments

In a similar research, [9] established comparison of ARIMA and LSTM and found that LSTM is more accurate than ARIMA in a shorter dataset of 36 data points and this result is reverted in longer dataset of 228 data points.

In this research, a vanilla LSTM was defined as an LSTM structure with preset hyperparameters i.e., epoch at 25, unit at 25, and the SGDM as optimiser. The experiments were then conducted by including variations of epoch and unit to be 125 and 300, and options of optimiser to include RMSProp and Adam. Further on the LSTM network architecture and training options were also described in Figure 3.

These experiments were performed using MatLab R2022b, experiment manager set at exhaustive sweep. The overall experiment steps were summarised in Figure 3.

IV. Result and Discussion

The results are reported in Table I, and comparison is visualised in Figure 4 and Figure 5. A further comparison of the effect of the number of epochs and selection of solver is visualised in Figure 6.

A. Number of Epochs

From the experiments on number of epochs, the result was showing a strong correlation of the number of epochs to the increase in prediction accuracy with highest epoch at 300

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on November 14,2024 at 15:15:09 UTC from IEEE Xplore. Restrictions apply.

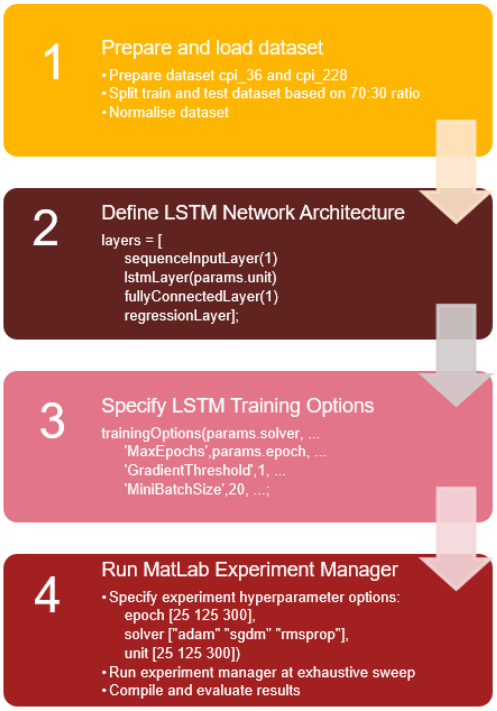

Fig. 3. Experiments steps

producing the highest prediction accuracy. The model is able to produce better results of 64% more accurate when the number of epochs is increased from 25 to 125, and 77% more accurate when the number epoch is increased from 25 to 300. However, the result also showed that there is a possibility of "plateau" as such the model can no longer produce significant increase in prediction accuracy as observed when the epoch is increased from 125 to 300 and as shown in Figure 5.

This result contradicts the finding of [10] where the number of epochs had no effect on the performance of the trained forecast model. However, it is also worth to note that there could be differences in the experiments as [10] also noted that a rolling mechanism, model is refined in each round and thus a whole new LSTM model is being trained, and thus the weights are updated each time for a new model, was used in the earlier research.

It is worth to note that the increase of accuracy from the increase in epoch is also at the cost of computing time albeit not very significant for the shorter dataset. The train time for 25 epoch and 300 epoch for the cpi_36 was 4 seconds and 5 seconds, or 25% more time needed. Meanwhile the train time for 25 epoch and 300 epoch for cpi_228 was 6 and 10 seconds, or 67% more time needed.

B. Selection of Optimiser or Solver

Upon the selection of optimiser or solver, SGDM is able to produce the most accurate result with RMSE at 0.51 compared to Adam and RMSProp with RMSE at 0.89 and 0.78, respectively. The author argues this is probably due to the nature of Adam where it combines the result of the first moment, that is close or similar to SGDM, with the result of the second moment, which is close or similar to AdaDelta and RMSProp, where the second moment produced inferior accuracy. If analogised as a ball rolling down on a slope, Adam behaved like a heavy ball due to friction, which might then prefer flat minima in the error surface [12], [23].

C. Number of Units

Results from the experiment with the number of units were inconclusive. The initial increase of unit from 25 to 125 yielded a relatively low accuracy improvement of 6% in RMSE. This result apparently worsened when the number of units was further increased to 300 by -2%, compared to the number of units at 125. This result is comparable with the findings of [11] that studied the effect of the number of LSTM units and found that an increased number of LSTM cells initially improves the prediction, and above a certain number causes poorer results.

D. Experiment on Longer Dataset

When tested with longer dataset of cpi_228, the model, as visualised in Figure 4 and Figure 6, produced a consistent result and trend over the impact of number of epochs, number of units, and selection of optimiser or solver. The author argued that this further confirms the analysis of results from the experiment on a shorter dataset cpi_36. It was also interesting to note that when compared of similar hyperparameters i.e., at epoch 300, unit 25, and using SGDM optimiser, the prediction accuracy of LSTM was actually decreasing from RMSE 0.1148 to 0.2832 on the dataset cpi_36 and cpi_228, respectively. This result is also consistent with the findings of [9], [20] that LSTM predictions are more satisfactory when a lesser time span of data is used as training data. The LSTM result worsens when a larger time span of data is used.

E. Combined Effect of Epochs and Units on SGDM

A further analysis into the effect of number of epochs and units on the SGDM was performed and visualised in Figure 6. It showed that the most accurate prediction result on dataset cpi_36 with RMSE at 0.1148 was on the setting of epoch at 300 and unit at 25. When compared with the vanilla LSTM, this is a significant increase of accuracy by 77%. Further tested on the longer dataset of cpi_228, the highest prediction accuracy with RMSE at 0.2524, was on the setting of epoch at 300 and unit at 300. This result further signifies the importance of setting a certain optimal value or number of epochs and units in tuning the LSTM for a better prediction accuracy.

F. Discussion

From this research results, there are hyperparameters that are more sensitive than others resulting in the prediction accuracy such as the number of epochs and selection of optimiser or solver than the number of units. This research also showed that hyperparameter tuning when combined does not necessarily improve the accuracy of prediction results. In all, the author found that it can be difficult to determine an optimal hyperparameter tuning without proper human interpretation.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on November 14,2024 at 15:15:09 UTC from IEEE Xplore. Restrictions apply.

TABLE I
RESULT COMPARISON

Trial	Epoch	Solver	Unit	RMSE (cpi_36)	RMSE (cpi_228)
1	25	adam	25	0.8929	0.8904
2	125	adam	25	0.309	0.3332
3	300	adam	25	0.1822	0.2688
4	25	sgdm	25	0.5056	0.4417
5	125	sgdm	25	0.1811	0.3167
6	300	sgdm	25	0.1148	0.2832
7	25	rmsprop	25	0.7821	0.8102
8	125	rmsprop	25	0.3205	0.3602
9	300	rmsprop	25	0.2348	0.299
10	25	adam	125	0.5462	0.4904
11	125	adam	125	0.2106	0.2578
12	300	adam	125	0.148	0.1797
13	25	sgdm	125	0.4757	0.423
14	125	sgdm	125	0.3747	0.3326
15	300	sgdm	125	0.2973	0.2796
16	25	rmsprop	125	0.5168	0.4668
17	125	rmsprop	125	0.3602	0.3306
18	300	rmsprop	125	0.2512	0.2402
19	25	adam	300	0.5198	0.4743
20	125	adam	300	0.2725	0.2588
21	300	adam	300	0.1142	0.1621
22	25	sgdm	300	0.4835	0.4339
23	125	sgdm	300	0.3987	0.3387
24	300	sgdm	300	0.3011	0.2524
25	25	rmsprop	300	0.5089	0.4672
26	125	rmsprop	300	0.3483	0.31
27	300	rmsprop	300	0.2105	0.2351

V. CONCLUSION AND FUTURE WORK

From the experiment results, it can be concluded that hyperparameters fine tuning especially in the number of epochs and selection of optimiser can significantly improve the prediction accuracy of LSTM. The number of units however showed an inconclusive result toward the prediction accuracy.

This research has analysed the effect of the number of epochs, units, and selection of optimiser as the hyperparameters configuration on the accuracy of prediction result. Several possible subjects for further research include expanding the study to other hyperparameters e.g., changes in learning rate, selection of activation function, customising structure of the LSTM cell, gate, or layer, and exploring another non-linear dataset.

ACKNOWLEDGMENT

This project has been supported by the Graduate Informatics Program, Faculty of Computer Science, Universitas Pelita Harapan, Indonesia.

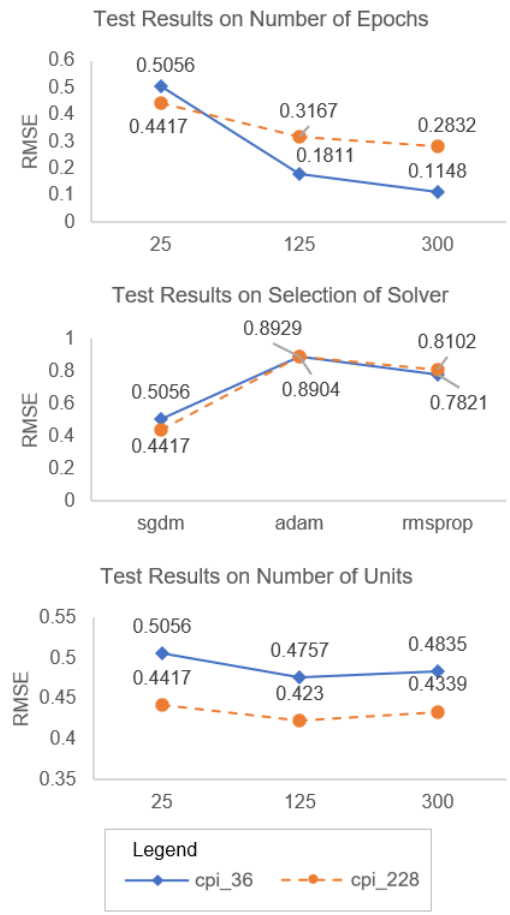

Fig. 4. Effect of Tuning on Prediction Accuracy and Different Dataset

REFERENCES

[1] V. Kotu and B. Deshpande, "Chapter 12 - time series forecasting," in *Data Science (Second Edition)*, V. Kotu and B. Deshpande, Eds. Morgan Kaufmann, 2019, pp. 395–445. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780128147610000125

[2] G. Mahalakshmi, S. Sridevi, and S. Rajaram, "A survey on forecasting of time series data," in *2016 International Conference on Computing Technologies and Intelligent Data Engineering (ICCTIDE'16)*, 2016, pp. 1–8.

[3] A. Jalil and N. H. Rao, "Chapter 8 - time series analysis (stationarity, cointegration, and causality)," in *Environmental Kuznets Curve (EKC)*, B. Özcan and I. Öztürk, Eds. Academic Press, 2019, pp. 85–99. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780128167977000084

[4] S. Özen, V. Atalay, and A. Yazici, "Comparison of predictive models for forecasting time-series data," in *Proceedings of the 2019 3rd International Conference on Big Data Research*, ser. ICBDR 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 172–176. [Online]. Available: https://doi.org/10.1145/3372454.3372482

[5] B. A. Kalinggo and Zulkarnain, "Time series forecasting for non-stationary data: A case study of petrochemical product price," in *Proceedings of the 3rd Asia Pacific Conference on Research in Industrial and Systems Engineering 2020*, ser. APCORISE 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 86–92. [Online]. Available: https://doi.org/10.1145/3400934.3400952

[6] P. T. Yamak, L. Yujian, and P. K. Gadosey, "A comparison between arima, lstm, and gru for time series forecasting," in *Proceedings of the 2019 2nd International Conference on Algorithms, Computing and Artificial Intelligence*, ser. ACAI 2019. New York, NY, USA:

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on November 14,2024 at 15:15:09 UTC from IEEE Xplore. Restrictions apply.

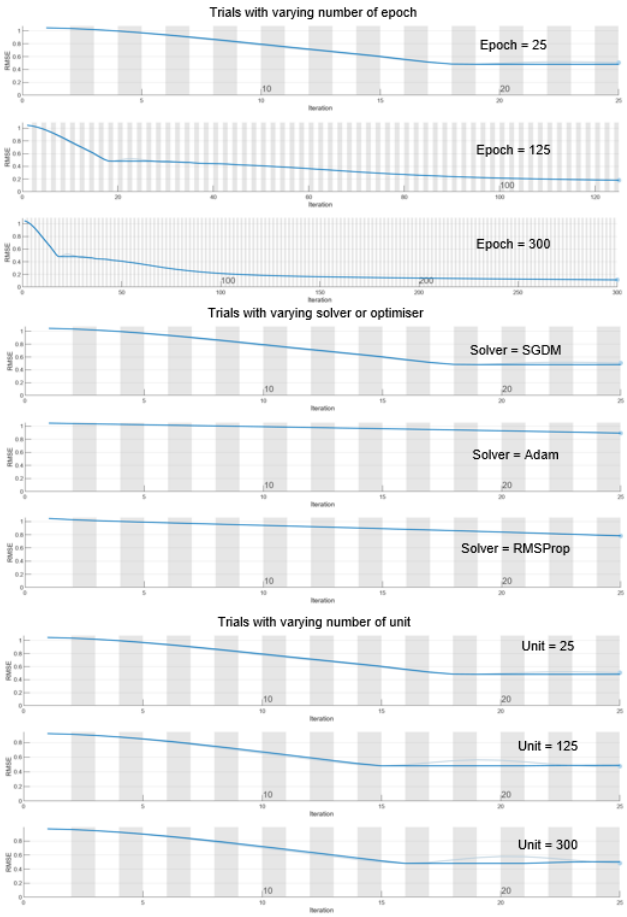

Fig. 5. Model Performance at Trials with Dataset cpi_36

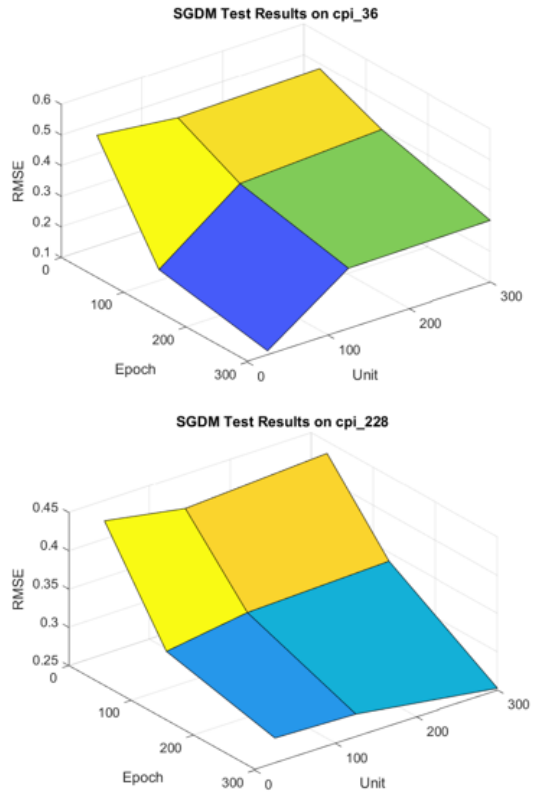

Fig. 6. Prediction Accuracy Comparison on the Effect of Number of Epochs and Units on SGDM Optimiser

Association for Computing Machinery, 2019, p. 49–55. [Online]. Available: https://doi.org/10.1145/3377713.3377722

[7] Z. Li, H. Zou, and B. Qi, "Application of arima and lstm in relative humidity prediction," in *2019 IEEE 19th International Conference on Communication Technology (ICCT)*, 2019, pp. 1544–1549.

[8] Z. Gao, "Stock price prediction with arima and deep learning models," in *2021 IEEE 6th International Conference on Big Data Analytics (ICBDA)*, 2021, pp. 61–68.

[9] D. G. Taslim and I. M. Murwantara, "A comparative study of arima and lstm in forecasting time series data," in *2022 9th International Conference on Information Technology, Computer, and Electrical Engineering (ICITACEE)*, 2022, pp. 231–235.

[10] S. Siami-Namini, N. Tavakoli, and A. Siami Namin, "A comparison of arima and lstm in forecasting time series," in *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2018, pp. 1394–1401.

[11] S. Chakraborty, J. Banik, S. Addhya, and D. Chatterjee, "Study of dependency on number of lstm units for character based text generation models," in *2020 International Conference on Computer Science, Engineering and Applications (ICCSEA)*, 2020, pp. 1–5.

[12] S. R. Dubey, S. Chakraborty, S. K. Roy, S. Mukherjee, S. K. Singh, and B. B. Chaudhuri, "diffgrad: An optimization method for convolutional neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, no. 11, pp. 4500–4511, 2020.

[13] E. M. Dogo, O. J. Afolabi, N. I. Nwulu, B. Twala, and C. O. Aigbavboa, "A comparative analysis of gradient descent-based optimization algorithms on convolutional neural networks," in *2018 International Conference on Computational Techniques, Electronics and Mechanical Systems (CTEMS)*, 2018, pp. 92–99.

[14] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," Jan 2017. [Online]. Available: https://arxiv.org/abs/1412.6980

[15] G. Box and G. Jenkins, *Time Series Analysis: Forecasting and Control*. Holden-Day, 1976.

[16] J. G. De Gooijer and R. J. Hyndman, "25 years of time series forecasting," *International Journal of Forecasting*, vol. 22, no. 3, pp. 443–473, 2006, twenty five years of forecasting. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0169207006000021

[17] A. A. Ariyo, A. O. Adewumi, and C. K. Ayo, "Stock price prediction using the arima model," in *2014 UKSim-AMSS 16th International Conference on Computer Modelling and Simulation*, 2014, pp. 106–112.

[18] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, p. 1735–1780, nov 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[19] B. Lindemann, T. Müller, H. Vietz, N. Jazdi, and M. Weyrich, "A survey on long short-term memory networks for time series prediction," *Procedia CIRP*, vol. 99, pp. 650–655, 2021, 14th CIRP Conference on Intelligent Computation in Manufacturing Engineering, 15-17 July 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2212827121003796

[20] B. Joosery and G. Deepa, "Comparative analysis of time-series forecasting algorithms for stock price prediction," in *Proceedings of the International Conference on Advanced Information Science and System*, ser. AISS '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: https://doi.org/10.1145/3373477.3373699

[21] P. Lovie, *Coefficient of Variation*. John Wiley Sons, Ltd, 2005, pp. 317–318. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/0470013192.bsa107

[22] R. Pelánek, "Measuring predictive performance of user models: The details matter." Association for Computing Machinery, Inc, 7 2017, pp. 197–201.

[23] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," 2018.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on November 14,2024 at 15:15:09 UTC from IEEE Xplore. Restrictions apply.